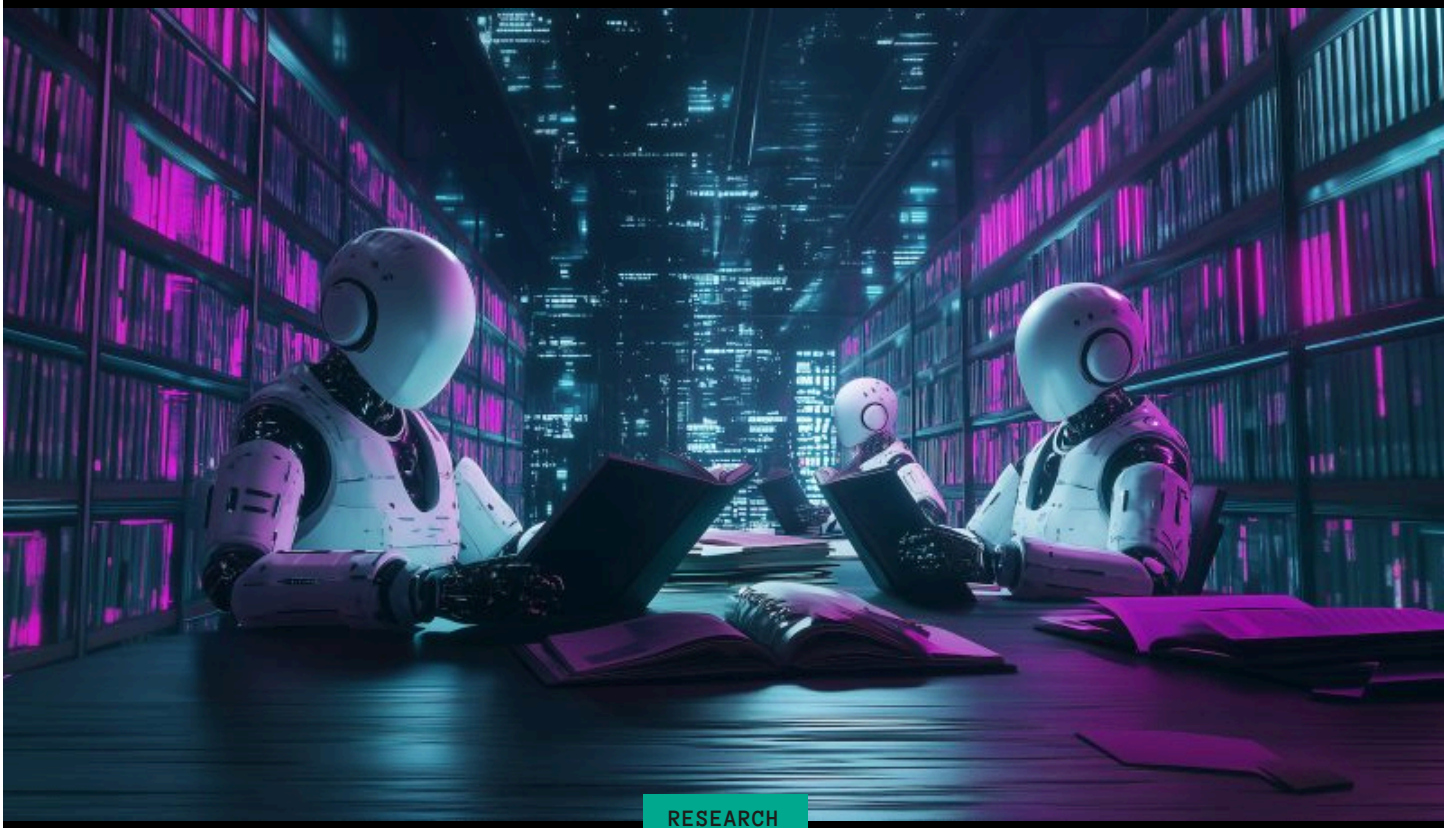

RESEARCH

# How we trained an ML model to detect DLL hijacking

06 OCT 2025 8 minute read

Expert ANNA PIDZHAKOVA

Table of Contents

DLL hijacking is a common technique in which attackers replace a library called by a legitimate process with a malicious one. It is used by both creators of mass-impact malware, like stealers and banking Trojans, and by APT and cybercrime groups behind targeted attacks. In recent years, the number of DLL hijacking attacks has grown significantly.

kaspersky

***Trend in the number of DLL hijacking attacks. 2023 data is taken as 100% (download)***

We have observed this technique and its variations, like DLL sideloading, in targeted attacks on organizations in Russia, Africa, South Korea, and other countries and regions. Lumma, one of 2025's most active stealers, uses this method for distribution. Threat actors trying to profit from popular applications, such as DeepSeek, also resort to DLL hijacking.

Detecting a DLL substitution attack is not easy because the library executes within the trusted address space of a legitimate process. So, to a security solution, this activity may look like a trusted process. Directing excessive attention to trusted processes can compromise overall system performance, so you have to strike a delicate balance between a sufficient level of security and sufficient convenience.

## Detecting DLL hijacking with a machine-learning model

Artificial intelligence can help where simple detection algorithms fall short. Kaspersky has been using machine learning for 20 years to identify malicious activity at various stages. The AI expertise center researches the capabilities of different models in threat detection, then trains and implements them. Our colleagues at the threat intelligence center approached us with a question of whether machine learning could be used to detect DLL hijacking, and more importantly, whether it would help improve detection accuracy.

## Preparation

To determine if we could train a model to distinguish between malicious and legitimate library loads, we first needed to define a set of features highly indicative of DLL hijacking. We identified the following key features:

- **Wrong library location.** Many standard libraries reside in standard directories, while a malicious DLL is often found in an unusual location, such as the same folder as the executable that calls it.
- **Wrong executable location.** Attackers often save executables in non-standard paths, like temporary directories or user folders, instead of %Program Files%.
- **Renamed executable.** To avoid detection, attackers frequently save legitimate applications under arbitrary names.
- **Library size has changed, and it is no longer signed.**
- **Modified library structure.**

## Training sample and labeling

For the training sample, we used dynamic library load data provided by our internal automatic processing systems, which handle millions of files every day, and anonymized telemetry, such as that voluntarily provided by Kaspersky users through Kaspersky Security Network.

The training sample was labeled in three iterations. Initially, we could not automatically pull event labeling from our analysts that indicated whether an event was a DLL hijacking attack. So, we used data from our databases containing only file reputation, and labeled the rest of the data manually. We labeled as DLL hijacking those library-call events where the process was definitively legitimate but the DLL was definitively malicious. However, this labeling was not enough because some processes, like "svchost", are designed mainly to load various libraries. As a result, the model we trained on this data had a high rate of false positives and was not practical for real-world use.

In the next iteration, we additionally filtered malicious libraries by family, keeping only those which were known to exhibit DLL-hijacking behavior. The model trained on this refined data showed significantly better accuracy and essentially confirmed our hypothesis that we could use machine learning to detect this type of attacks.

At this stage, our training dataset had tens of millions of objects. This included about 20 million clean files and around 50,000 definitively malicious ones.

| Status | Total | Unique files |
|---|---|---|
| Unknown | ~ 18M | ~ 6M |
| Malicious | ~ 50K | ~ 1,000 |
| Clean | ~ 20M | ~ 250K |

We then trained subsequent models on the results of their predecessors, which had been verified and further labeled by analysts. This process significantly increased the efficiency of our training.

## Loading DLLs: what does normal look like?

So, we had a labeled sample with a large number of library loading events from various processes. How can we describe a "clean" library? Using a process name + library name combination does not account for renamed processes. Besides, a legitimate user, not just an attacker, can rename a process. If we used the process hash instead of the name, we would solve the renaming problem, but then every version of the same library would be treated as a separate library. We ultimately settled on using a library name + process signature combination. While this approach considers all identically named libraries from a single vendor as one, it generally produces a more or less realistic picture.

To describe safe library loading events, we used a set of counters that included information about the processes (the frequency of a specific process name for a file with a given hash, the frequency of a specific file path for a file with that hash, and so on), information about the

libraries (the frequency of a specific path for that library, the percentage of legitimate launches, and so on), and event properties (that is, whether the library is in the same directory as the file that calls it).

The result was a system with multiple aggregates (sets of counters and keys) that could describe an input event. These aggregates can contain a single key (e.g., a DLL's hash sum) or multiple keys (e.g., a process's hash sum + process signature). Based on these aggregates, we can derive a set of features that describe the library loading event. The diagram below provides examples of how these features are derived:

*Feature extraction from aggregates*

## Loading DLLs: how to describe hijacking

Certain feature combinations (dependencies) strongly indicate DLL hijacking. These can be simple dependencies. For some processes, the clean library they call always resides in a separate folder, while the malicious one is most often placed in the process folder.

Other dependencies can be more complex and require several conditions to be met. For example, a process renaming itself does not, on its own, indicate DLL hijacking. However, if the new name appears in the data stream for the first time, and the library is located on a non-standard path, it is highly likely to be malicious.

## Model evolution

Within this project, we trained several generations of models. The primary goal of the first generation was to show that machine learning could at all be applied to detecting DLL hijacking. When training this model, we used the broadest possible interpretation of the term.

The model's workflow was as simple as possible:

1. We took a data stream and extracted a frequency description for selected sets of keys.
2. We took the same data stream from a different time period and obtained a set of features.
3. We used type 1 labeling, where events in which a legitimate process loaded a malicious library from a specified set of families were marked as DLL hijacking.
4. We trained the model on the resulting data.

*First-generation model diagram*

The second-generation model was trained on data that had been processed by the first-generation model and verified by analysts (labeling type 2). Consequently, the labeling was more precise than during the training of the first model. Additionally, we added more features to describe the library structure and slightly complicated the workflow for describing library loads.

*Second-generation model diagram*

Based on the results from this second-generation model, we were able to identify several common types of false positives. For example, the training sample included potentially unwanted applications. These can, in certain contexts, exhibit behavior similar to DLL hijacking, but they are not malicious and rarely belong to this attack type.

We fixed these errors in the third-generation model. First, with the help of analysts, we flagged the potentially unwanted applications in the training sample so the model would not detect them. Second, in this new version, we used an expanded labeling that included useful detections from both the first and second generations. Additionally, we expanded the feature description through one-hot encoding — a technique for converting categorical features into a binary format — for certain fields. Also, since the volume of events processed by the model increased over time, this version added normalization of all features based on the data flow size.

*Third-generation model diagram*

## Comparison of the models

To evaluate the evolution of our models, we applied them to a test data set none of them had worked with before. The graph below shows the ratio of true positive to false positive verdicts for each model.

*Trends in true positives and false positives from the first-, second-, and third-generation models*

As the models evolved, the percentage of true positives grew. While the first-generation model achieved a relatively good result (0.6 or higher) only with a very high false positive rate ($10^{-3}$ or more), the second-generation model reached this at $10^{-5}$. The third-generation model, at the same low false positive rate, produced 0.8 true positives, which is considered a good result.

Evaluating the models on the data stream at a fixed score shows that the absolute number of new events labeled as DLL Hijacking increased from one generation to the next. That said, evaluating the models by their false verdict rate also helps track progress: the first model has a fairly high error rate, while the second and third generations have significantly lower ones.

***False positives rate among model outputs, July 2024 – August 2025 (download)***

## Practical application of the models

All three model generations are used in our internal systems to detect likely cases of DLL hijacking within telemetry data streams. We receive 6.5 million security events daily, linked to 800,000 unique files. Aggregates are built from this sample at a specified interval, enriched, and then fed into the models. The output data is then ranked by model and by the probability of DLL hijacking assigned to the event, and then sent to our analysts. For instance, if the third-generation model flags an event as DLL hijacking with high confidence, it should be investigated first, whereas a less definitive verdict from the first-generation model can be checked last.

Simultaneously, the models are tested on a separate data stream they have not seen before. This is done to assess their effectiveness over time, as a model's detection performance can degrade. The graph below shows that the percentage of correct detections varies slightly over time, but on average, the models detect 70–80% of DLL hijacking cases.

***DLL hijacking detection trends for all three models, October 2024 – September 2025 (download)***

Additionally, we recently deployed a DLL hijacking detection model into the Kaspersky SIEM, but first we tested the model in the Kaspersky MDR service. During the pilot phase, the model helped to detect and prevent a number of DLL hijacking incidents in our clients' systems. We have written a separate article about how the machine learning model for detecting targeted attacks involving DLL hijacking works in Kaspersky SIEM and the incidents it has identified.

## Conclusion

Based on the training and application of the three generations of models, the experiment to detect DLL hijacking using machine learning was a success. We were able to develop a model that distinguishes events resembling DLL hijacking from other events, and refined it to a state suitable for practical use, not only in our internal systems but also in commercial products. Currently, the models operate in the cloud, scanning hundreds of thousands of unique files per month and detecting thousands of files used in DLL hijacking attacks each month. They regularly

identify previously unknown variations of these attacks. The results from the models are sent to analysts who verify them and create new detection rules based on their findings.

SECURITY TECHNOLOGY | MACHINE LEARNING | DLL HIJACKING | THREAT HUNTING | ARTIFICIAL INTELLIGENCE | DLL

## How we trained an ML model to detect DLL hijacking

Your email address will not be published. Required fields are marked *

Type your comment here

Name *

Email *

Comment

This site uses Akismet to reduce spam. Learn how your comment data is processed.

## // LATEST POSTS

**IT threat evolution in Q1 2026. Non-mobile statistics**

AMR

**Kimsuky targets organizations with PebbleDash-based tools**

SOJUN RYU

**State of ransomware in 2026**

FABIO ASSOLINI, MARC RIVERO, MAHER YAMOUT, DARYA GORODILOVA

**CVE-2025-68670: discovering an RCE vulnerability in xrdp**

DENIS SKVORTSOV, DMITRY SHMOYLOV

## // LATEST WEBINARS

TRAININGS AND WORKSHOPS

05 MAY 2026, 5:00PM 47 MIN

**How do we catch 0-days?**

BORIS LARIN

TRAININGS AND WORKSHOPS

30 APR 2026, 2:00PM 61 MIN

**SOC evolution: From firefighting to strategic risk management**

ROMAN NAZAROV, ALEXANDER MAZIKIN, ALEXANDER MARMALIDI

TECHNOLOGIES AND SERVICES

16 APR 2026, 1:00PM 119 MIN

**The power of enrichment: Boosting your product with threat intelligence**

DMITRY TAYGIND, OLEG SHABUROV

CYBERTHREAT TALKS

24 MAR 2026, 4:00PM 68 MIN

**2026 cyber world: Inside the attacker's playbook**

SERGEY SOLDATOV, KONSTANTIN SAPRONOV

# // REPORTS

**Cloud Atlas activity in the second half of 2025 and early 2026: new tools and a new payload**

Cloud Atlas attacks the public sector and diplomatic structures of Russia and Belarus, using ReverseSocks, SSH, and Tor for persistence in infected systems and its new tool, PowerCloud.

**Kimsuky targets organizations with PebbleDash-based tools**

**OceanLotus suspected of using PyPI to deliver ZiChatBot malware**

**Silver Fox uses the new ABCDoor backdoor to target organizations in Russia and India**



Threats

Categories

**Archive**
**Webinars**
**Statistics**
**Threats descriptions**
**Kaspersky ICS CERT**

**All tags**
**APT Logbook**
**Encyclopedia**
**KSB 2025**

kaspersky